



Software Measurement and Metrics

Project

Submitted By:

**Kashaf Fatima (22i-2415)
M Safi ur Rehman (22i-2534)
Amna Noor (22i-1529)
SE-8F**

Submitted to:

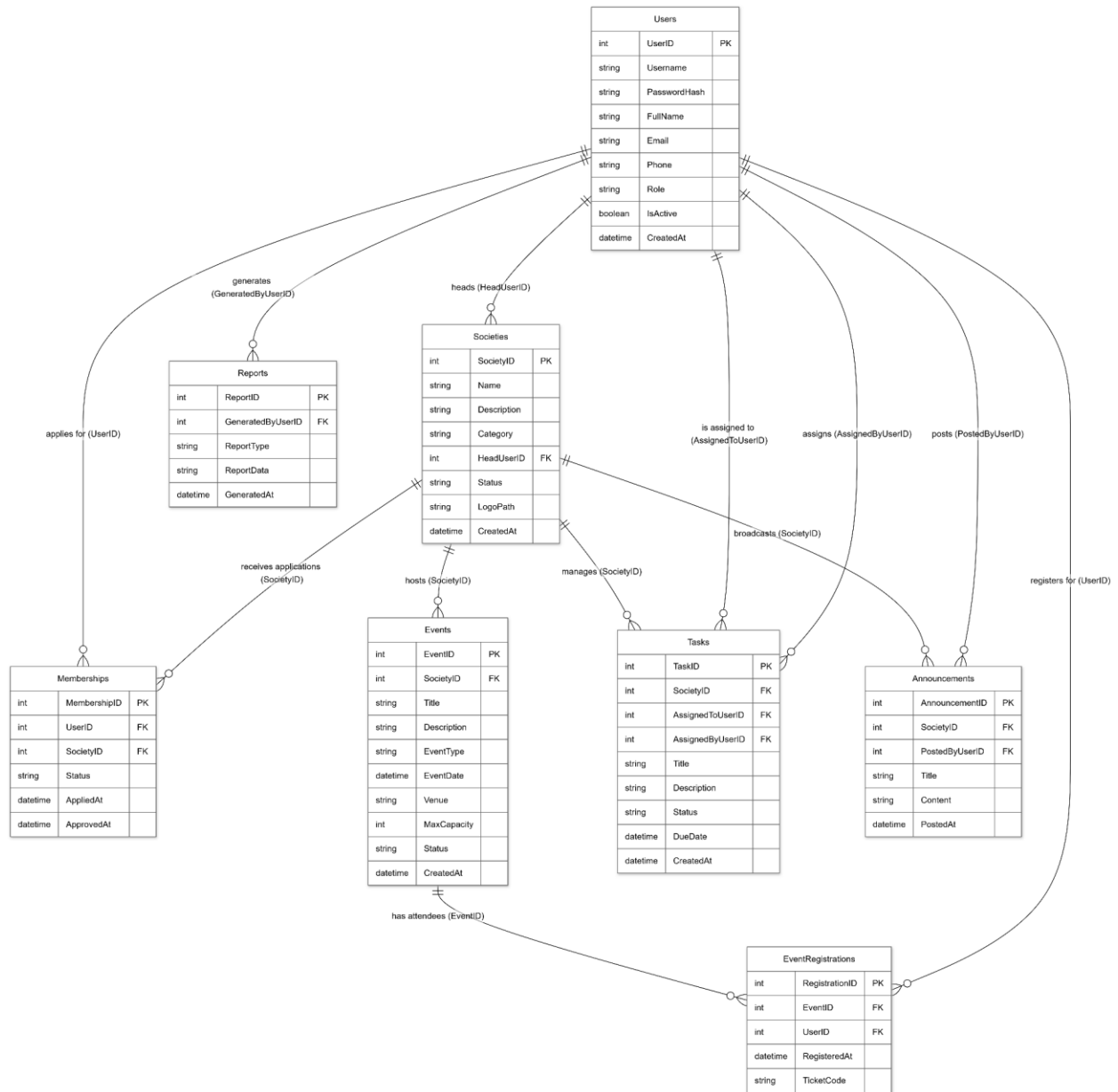
Mr. Bilal Khalid Dar

Table of Contents

ERD	4
Student 1	5
Task 2:.....	5
Cyclomatic complexity and Test case inputs	5
Task 5:.....	17
Automated Test Cases Executed	17
Faults injection	19
Fault Injection Execution Results.....	26
Reliability Analysis Table.....	27
Most Reliable Module	28
Least Reliable Module	28
Student 2	28
Task 3	28
Module 1: Admin	29
Module 2: Society	30
Module 3: Student.....	31
Module 4: Forms	32
Modules 5: Helpers	33
Comparison Table	33
Task 4: Apply CK metrics on the complete generated project.	35
WMC – Weighted Methods per Class	35
DIT – Depth of Inheritance Tree.....	40
NOC – Number of Children	42
CBO – Coupling Between Objects	44
RFC – Response for Class	46
LCOM – Lack of Cohesion in Methods.....	50
Analysis summary and findings	55
Task 9 : Refactoring Suggestions for the Student Management System	56

1. Helper module	56
2. Forms module	56
Overall Architecture Recommendations	57
Student 3	57
Task 7: Cocomo Model:	57
.cs Files	57
.Designer.cs Files	58
Task 8: Documentation ratio:	59
Task 6: KLM analysis:	59
Login Screen:	59
Admin Dashboard:	59
Manage students:	60
Manage Societies:	60
University Reports:	61
Society Head Dashboard:	61
Manage Members:	61
Manage Events:	62
Assign tasks:	63
Mark complete:	64
Reports:	64
Student Dashboard:	64
Browse Societies:	65
My Memberships:	65
My Tickets:	65
Upcoming Events:	66
Sign up:	66

ERD



Student 1

Task 2:

Perform Cyclomatic Complexity for all functions and write test cases for each of the function.

Deliverable: Table with name of function, cyclomatic complexity and test case inputs

Cyclomatic complexity and Test case inputs

Function Name	File Name	Cyclomatic Complexity	Test case Input
Main ()	Program.cs	1	No user input needed, Runs by itself as soon we click on run button
LoginForm()	LoginForm.cs	1	No user input is needed, When main will load the form
btnLogin_Click	LoginForm.cs	8	Valid case : Already registered username, correct password,(e.g., username = "admin" , password = 123) Click on button
btnLogin_Click	LoginForm.cs	8	Invalid case 1 : Wrong username, correct password, (e.g., username = "newuser" , password = 123), click on button
btnLogin_Click	LoginForm.cs	8	Invalid case 2 : Correct username, Wrong password, (e.g., username = "admin" , password = 1234), click on button
btnLogin_Click	LoginForm.cs	8	Invalid case 3 : Empty fields, (e.g., username = "" , password = "") click

			on button
InkRegister_LinkClicked	LoginForm.cs	1	Click on link (link click event)
HashPassword	LoginForm.cs	2	Valid case : Enter correct password
HashPassword	LoginForm.cs	2	Invalid case 1: Enter wrong password
HashPassword	LoginForm.cs	2	Invalid case 2 : Leave empty
RegisterForm	RegisterForm.cs	1	User clicks on the link on login page, registerform will be initialized
btnRegister_Click	RegisterForm.cs	5	Valid case : Enter all correct credentials (e.g., Full username, email, password, phone, Full Name) Click on Register button
btnRegister_Click	RegisterForm.cs	5	Invalid case 1 : Leave all fields empty, click on Register button
btnRegister_Click	RegisterForm.cs	5	Invalid case 2 : Different password and confirm password, click on Register button
btnRegister_Click	RegisterForm.cs	5	Invalid case 3 : Leave any on field empty , click on register button
btnRegister_Click	RegisterForm.cs	5	Invalid case 4 : Enter username which is already registered, click on Register button

InkLogin_LinkClicked	RegisterForm.cs	1	Click on link to close the form
AdminDashboard	AdminDashboard.cs	1	Click on Login button after entering admin credentials No specific user input required for it, it will initialize the form
AdminDashboard_Load	AdminDashboard.cs	1	Loads the dashboard, no specific user input is required
LoadStats	AdminDashboard.cs	1	No specific user input is required, loads the information like students, societies, events etc (DB query execution)
btnStudents_Click	AdminDashboard.cs	1	Manage student Button click
btnSocieties_Click	AdminDashboard.cs	1	Manage Societies button click
btnLogout_Click	AdminDashboard.cs	1	Logout button click
btnEvents_Click	AdminDashboard.cs	1	Approve events button click
btnReports_Click	AdminDashboard.cs	1	University Report Management button click
ApproveEventsForm	ApproveEventsForm.cs	1	No user input is required, after navigating the event form, this function will automatically initializes the form by itself
ApproveEventsForm_Load	ApproveEventsForm.cs	1	No user input is required, it will load the events by itself
LoadEvents	ApproveEventsForm.cs	2	No specific user input is required, all events will be loaded on

			event forms
btnApprove_Click	ApproveEventsForm.cs	2	Select the event Click the Approve button
btnReject_Click	ApproveEventsForm.cs	2	Select the event Click the Reject button
btnClose_Click	ApproveEventsForm.cs	1	Click on close (x) button
ManageSocietiesForm	ManageSocietiesForm	1	No user input is required, after navigating the societies form, this function will automatically initializes the form by itself
LoadSocieties	ManageSocietiesForm	2	No specific user input is required, all societies will be loaded on the forms
UpdateStatus	ManageSocietiesForm	2	Valid case : Select the society, and click on delete button
UpdateStatus	ManageSocietiesForm	2	Invalid case : don't select any society, click on manage society
btnApprove_Click	ManageSocietiesForm	1	Click on Approve button
btnSuspend_Click	ManageSocietiesForm	1	Click on Suspend button
btnDelete_Click	ManageSocietiesForm	2	Click on delete button, Confirm to yes
btnDelete_Click	ManageSocietiesForm	2	Click on delete button, Confirm to no
btnClose_Click	ManageSocietiesForm	1	Click on close (x)

	m		button
ManageSocietiesForm_Load	ManageSocietiesForm	1	No user input is required, it will load the societies by itself
ManageStudentsForm	ManageStudentsForm	1	No user input is required, after navigating the students form, this function will initialize the form by itself
ManageStudentsForm_Load	ManageStudentsForm	1	No user input is required, loads the students data
LoadStudents	ManageStudentsForm	3	No user input is required, load the students data and arrange them in order
btnSearch_Click	ManageStudentsForm	1	Click on search button
btnActivate_Click	ManageStudentsForm	2	Select student and click on Activate button
btnActivate_Click	ManageStudentsForm	2	Don't select student and click on Activate button
btnDeactivate_Click	ManageStudentsForm	2	Select student and click on Deactivate button
btnDeactivate_Click	ManageStudentsForm	2	Don't select student and click on Deactivate button
btnClose_Click	ManageStudentsForm	1	Click on close (x) button
UniversityReportsForm	UniversityReportsForm	1	No user input is required, after navigating the reports form by clicking on university reports, this function will initialize

			the form by itself
btnStudentReport_Click	UniversityReportsForm	3	Click on button Students
btnSocietyReport_Click	UniversityReportsForm	1	Click on button Societies
btnEventReport_Click	UniversityReportsForm	1	Click on button Events
btnMembershipReport_Click	UniversityReportsForm	1	Click on button Memberships
btnClose_Click	UniversityReportsForm	1	Click on close (x) button
AssignTasksForm	AssignTasksForm	1	No user input is required, after navigating the reports form by clicking on assign tasks, this function will initialize the form by itself
AssignTasksForm_Load	AssignTasksForm	1	No user input is required, it will load the members of societies by itself, after form initialized
LoadMembers	AssignTasksForm	1	No user input is required, it will load the members of societies by itself
LoadTasks	AssignTasksForm	2	Select the valid society which has any task assigned
LoadTasks	AssignTasksForm	2	Select the invalid society which doesn't have any task assigned
btnAssign_Click	AssignTasksForm	2	Enter valid input fields (e.g., txtTaskTitle = "Prepare report", cmbMember.Selected Value = 3,

			txtTaskDesc = "Monthly report", valid dtpDue)
btnAssign_Click	AssignTasksForm	2	Invalid case 1: Leave the task title field empty and add other input fields as above (e.g., txtTaskTitle = "", cmbMember.Selected Value = 3)
btnAssign_Click	AssignTasksForm	2	Invalid case 2: Leave the member field empty (don't select any member) and add other input fields as above (e.g., txtTaskTitle = "Prepare report", cmbMember.Selected Value = null)
btnAssign_Click	AssignTasksForm	2	Invalid case 3 : Leave both fields empty (title and member fields) e.g., (txtTaskTitle = "", cmbMember.Selected Value = null)
btnMarkComplete_Click	AssignTasksForm	2	Select the task and click on Mark Complete button
btnMarkComplete_Click	AssignTasksForm	2	Don't select the task and click on Mark Complete button
btnClose_Click	AssignTasksForm	1	Click on close (x) button
ManageEventsForm	ManageEventsForm	1	No user input is required, after navigating the reports form by clicking on assign tasks, this function will initializes the form by itself

ManageEventsForm_Load	ManageEventsForm	1	No user input is required, it will load the events by itself, after form initialized
LoadEvents	ManageEventsForm	2	No user input is required, it will load the events by itself, after form initialized
btnAdd_Click	ManageEventsForm	2	Enter valid event details (e.g., Title = "Annual Meetup", Venue = "Main Hall", other fields valid), click on create event
btnAdd_Click	ManageEventsForm	2	Invalid case 1 : Leave the title field empty (e.g., Title = "", Venue = "Main Hall"), click on create event
btnAdd_Click	ManageEventsForm	2	Invalid case 2 : Leave the venue empty e.g., (Title = "Annual Meetup", Venue = ""), click on create event
btnAdd_Click	ManageEventsForm	2	Invalid case 3 : Leave both fields empty (e.g., Title = "", Venue = ""), click on create event
btnCancel_Click	ManageEventsForm	2	Select event and click on cancel event button
btnCancel_Click	ManageEventsForm	2	No event selected and click on cancel event button
ClearFields	ManageEventsForm	1	Click on Clear event button (x)
btnClose_Click	ManageEventsForm	1	Click on close (x) button
ManageMembersForm	ManageMembersForm	1	No user input required, form

			initializer (after user clicks on manage members)
ManageMembersForm_Load	ManageMembersForm	1	No user input required, form loads members by itself from DB
LoadPending	ManageMembersForm	2	Select society and click on Pending requests
LoadApproved	ManageMembersForm	2	Select society and click on approved requests
btnApprove_Click	ManageMembersForm	2	Select 1 pending member and click on approve button
btnApprove_Click	ManageMembersForm	2	Don't select any pending member and click on approve button
btnReject_Click	ManageMembersForm	2	Select 1 pending member and click on reject button
btnReject_Click	ManageMembersForm	2	Don't select any pending member and click on reject button
btnRemove_Click	ManageMembersForm	3	Select the member and confirm yes
btnRemove_Click	ManageMembersForm	3	Select the member and confirm no
btnClose_Click	ManageMembersForm	1	Click on close (x) button
SocietyDashboard	SocietyDashboard	1	Form Initialization (no specific user input is required)
SocietyDashboard_Load	SocietyDashboard	1	valid user with assigned society opens dashboard

LoadStats	SocietyDashboard	2	When Society with members/events/pending requests is opened
LoadStats	SocietyDashboard	2	Society with no records is selected
btnMembers_Click	SocietyDashboard	1	Click Manage Members button
btnEvents_Click	SocietyDashboard	1	Click Manage Events button
btnTasks_Click	SocietyDashboard	1	Click Assign Tasks button
btnReports_Click	SocietyDashboard	1	Click Reports button
btnLogout_Click	SocietyDashboard	1	Click Logout button
SocietyReportsForm	SocietyReportsForm	1	Form Initialization (no specific user input is required)
btnMemberReport_Click	SocietyReportsForm	1	Select society with members , click on Member Report button
btnMemberReport_Click	SocietyReportsForm	1	Select society with no members, click on Member Report button
btnEventReport_Click	SocietyReportsForm	1	Select society with events, click on Event Report button
btnEventReport_Click	SocietyReportsForm	1	Select society with no events, click on Event Report button
btnClose_Click	SocietyReportsForm	1	Click on close (x) button
BrowseSocietiesForm	BrowseSocietiesForm	1	Form Initialization (no specific user input is required)
BrowseSocietiesForm_Load	BrowseSocietiesForm	1	Open form with approved societies

LoadSocieties	BrowseSocietiesForm	3	Enter empty search text, click on search button
LoadSocieties	BrowseSocietiesForm	3	Enter existing society name
btnSearch_Click	BrowseSocietiesForm	1	Enter valid search term in search field
btnSearch_Click	BrowseSocietiesForm	1	Enter with empty search field
btnApply_Click	BrowseSocietiesForm	4	Select valid society, click on Apply for Membership button
btnApply_Click	BrowseSocietiesForm	4	Don't select any society and click on Apply for Membership button
MyMembershipsForm	MyMembershipsForm	1	Form Initialization (no specific user input is required)
MyMembershipsForm_Load	MyMembershipsForm	1	Open form for user with memberships, click on My Memberships button
MyMembershipsForm_Load	MyMembershipsForm	1	Open form for user with no memberships, click on My Memberships button
LoadData	MyMembershipsForm	2	Open for user with one membership, click on My Memberships button
LoadData	MyMembershipsForm	2	Open for user with no membership, click on My Memberships button
MyTicketsForm	MyTicketsForm	1	Form Initialization (no specific user input is required)
MyTicketsForm_Load	MyTicketsForm	1	Open form for user

			with tickets, click on My Tickets button
MyTicketsForm_Load	MyTicketsForm	1	Open form for user with no tickets, click on My Tickets button
StudentDashboard	StudentDashboard	1	Form Initialization (no specific user input is required)
StudentDashboard_Load	StudentDashboard	1	Login with valid student credentials
StudentDashboard_Load	StudentDashboard	1	Login with student credential with no memberships/events
LoadStats	StudentDashboard	1	Login with student credential with no memberships/events
LoadStats	StudentDashboard	1	Login with student credential with memberships/events/pending requests
btnBrowse_Click	StudentDashboard	1	Click browse society and apply for society
btnMemberships_Click	StudentDashboard	1	Click on My Memberships button
btnEvents_Click	StudentDashboard	1	Click Upcoming Events button and apply for event
btnTickets_Click	StudentDashboard	1	Click My Tickets button
btnLogout_Click	StudentDashboard	1	Click Logout button
UpcomingEventsForm	UpcomingEventsForm	1	Form Initialization (no specific user input is required)
UpcomingEventsForm_Load	UpcomingEventsForm	1	Open form with upcoming events
UpcomingEventsForm_Load	UpcomingEventsForm	1	Open form with no upcoming events

LoadEvents	UpcomingEventsForm	2	Valid events loaded
btnRegister_Click	UpcomingEventsForm	4	Select Valid event, click on Register for Event
btnRegister_Click	UpcomingEventsForm	4	Don't select any event and click on Register for Event button
GetConnection	DatabaseHelper	1	Valid connection String
ExecuteQuery	DatabaseHelper	2	Valid query Or Invalid SQL query
ExecuteNonQuery	DatabaseHelper	2	Valid insert/update Or Invalid SQL query
ExecuteScalar	DatabaseHelper	2	Valid query Or Invalid SQL query
Login	SessionManager	1	Valid user data
SetSociety	SessionManager	1	Valid society data
Logout	SessionManager	1	Click on logout

Task 5:

Inject 5 faults into each function/module of the generated application. Assume the confidence threshold (E) for all functions is 1.

Deliverable. A table with list of all features with number of faults identified and probability (no more than E fault). Use test cases generate after cyclomatic complexity

After the table, identify:

- Most Reliable Function/Feature (highest probability of having no more than 1 fault)
- Least Reliable Function/Feature (lowest probability of having no more than 1 fault)

Automated Test Cases Executed

Before injecting the file, I got the automated test cases for the each module which are as follow:

1) Authentication Module:

- a. TC_AUTH_01_ValidLogin: This testcase verifies that the login is successful and user is directed to correct dashboard.

- b. TC_AUTH_02_InvalidLogin_WrongPassword: This testcase verifies that if we enter wrong password, login should fail.
- c. TC_AUTH_02_InvalidLogin_EmptyFields: This testcase verifies empty fields to trigger validation errors.
- d. TC_AUTH_03_Registration_ValidAndInvalid: This testcase verifies correct fields for correct account creation.
- e. TC_AUTH_04_Registration_MismatchedPasswords: This testcase makes sure that the mismatched password halts the registration process.

2) Admin Module:

- a. TC_ADMIN_01_DashboardStatistics: It verifies the university-wide stats query logic without causing any crash.
- b. TC_ADMIN_02_ManageStudents_Search: It checks the student search and status update logic.
- c. TC_ADMIN_03_ManageSocieties_Approve: It checks that row selection bounds and society approval queries.

3) Society Module:

- a. TC_SOC_01_DashboardStatistics: It makes sure that the specific society statistics are loaded properly.
- b. TC_SOC_03_ManageEvents_InvalidAdd: It makes sure that the empty title/venue halts the process of event creation.

4) Student Module:

- a. TC_STU_01_DashboardStatistics: This testcase verifies the accurate display of student memberships and events.
- b. TC_STU_02_BrowseSocieties_Search: It checks the society search and membership application logic
- c. TC_STU_04_UpcomingEvents_Register: This testcase verifies the event registration logic and boundary checks

5) Core & Helper Module:

- a. TC_CORE_01_ExecuteQuery_Valid: It verifies the DB connection and query execution.
- b. TC_CORE_02_SessionManager_LoginLogout: This testcase verifies that the session state variable are correctly set and wiped.

When above testcases were run on the correct code, following were the result obtained:

- Total testcases : 15
- Passed : 15
- Failed : 0

```
..\Fault_Injected_Code > dotnet test SocietiesManagementSystem.Tests\SocietiesManagementSystem.Tests.csproj
g\net48\SocietiesManagementSystem.Tests.dll
Test run for D:\Documents\Sem_8\SMM\Project\Fault_Injected_Code\SocietiesManagementSystem.Tests\bin\Debug\net48\SocietiesManagementSystem.Tests.dll (.NETFramework,Version=v4.8)
VSTest version 17.14.1 (x64)

Starting test execution, please wait...
A total of 1 test files matched the specified pattern.

Passed! - Failed:    0, Passed:    15, Skipped:    0, Total:    15, Duration: 6 s - SocietiesManagementSystem.Tests.dll (net48)
```

Faults injection

After above steps faults were injected in the code. The details for the faults injection are as follow:

1) *Authentication Module*

- a. LoginForm: Injected the fault by changing the username and password check for empty string to only check username and allow empty password.

```
if (string.IsNullOrEmpty(username))
{
    MessageBox.Show("Please enter both username and password.", "Validation")
    return;
}
```

- b. LoginForm: Fault injected was that "Admin" in switch case was replaced by "admin", so that admin is considered as student.

```
switch (role)
{
    case "admin":
        dashboard = new AdminDashboard();
        break;
    case "SocietyHead":
```

- c. RegisterForm: Removing the confirm password check so that mismatched passwords are allowed by code.

```
// if (password != confirm)
// {
//     MessageBox.Show("Passwords do not match.", "Validation", MessageBoxButtons.OK);
//     return;
// }
```

- d. RegisterForm: Changing the check from >0 to >1 to allow duplicate users.

```
var exists = DatabaseHelper.ExecuteScalar("SELECT COUNT(*) FROM Users WHERE Username=@Username");
if (Convert.ToInt32(exists) > 1)
{
    MessageBox.Show("Username already exists.", "Duplicate", MessageBoxButtons.OK);
    return;
}
```

- e. RegisterForm: For default registration role, it was changed from student to Admin.

```
DatabaseHelper.ExecuteNonQuery("INSERT INTO Users (Username, PasswordHash, FullName, Email, Phone, Role) VALUES (@Username, @PasswordHash, @FullName, @Email, @Phone, 'Admin')",
    new SqlParameter("@Username", username), new SqlParameter("@PasswordHash", passwordHash),
    new SqlParameter("@FullName", fullName), new SqlParameter("@Email", email), new SqlParameter("@Phone", phone));
```

2) Admin Module

- a. AdminDashboard: Changing pending events query to count Approved events instead.

```
DatabaseHelper.ExecuteScalar("SELECT COUNT(*) FROM Events WHERE Status='Approved'").ToString();
DatabaseHelper.ExecuteScalar("SELECT COUNT(*) FROM Events WHERE Status='Approved'").ToString();
```

- b. ManageSocietiesForm: For fault injection I changed row selection check from =0 to < 0, bypassing the safety check and crashing on action.

```
private void UpdateStatus(string status)
{
    if (dgvSocieties.SelectedRows.Count < 0) { MessageBox.Show("Select a society."); return; }
    int id = Convert.ToInt32(dgvSocieties.SelectedRows[0].Cells["SocietyID"].Value);
    DatabaseHelper.ExecuteNonQuery("UPDATE Societies SET Status=@St WHERE SocietyID=@ID",
        new SqlParameter("@St", status), new SqlParameter("@ID", id));
    MessageBox.Show("Society " + status.ToLower() + " updated.", "Done", MessageBoxButtons.OK);
    LoadSocieties();
}
```

- c. ManageStudentsForm: Changing account activation SQL to set IsActive=0 instead of 1.

```
if (dgvStudents.SelectedRows.Count == 0) return;
int id = Convert.ToInt32(dgvStudents.SelectedRows[0].Cells["UserID"].Value);
DatabaseHelper.ExecuteNonQuery("UPDATE Users SET IsActive=0 WHERE UserID=@ID", ne
MessageBox.Show("Account activated.", "Done", MessageBoxButtons.OK, MessageBoxIcon
LoadStudents());
```

- d. ManageStudentsForm: Removing wildcards (%) from the SQL search query, breaking partial search.

```
dgvStudents.DataSource = string.IsNullOrEmpty(search)
    ? DatabaseHelper.ExecuteQuery(query)
    : DatabaseHelper.ExecuteQuery(query, new SqlParameter("@S", search));
if (dgvStudents.Columns.Contains("UserID")) dgvStudents.Columns["UserID"].Visibl
```

- e. UniversityReportsForm: For fault injection I changed the student report query to filter by Role="Admin".

```
IsActive, CreatedAt FROM Users WHERE Role='Admin' ORDER BY FullName");
```

3) Society Module

- a. ManageEventsForm: Changing null input validation from || to && to allow partially empty submissions.

```
if (string.IsNullOrWhiteSpace(txtTitle.Text) && string.IsNullOrWhiteSpace(txtVenue.Text)
{ MessageBox.Show("Title and Venue are required.", "Validation", MessageBoxButtons.OK,
```

- b. SocietyDashboard: Removing societyID check to risk null reference errors.

```
private void LoadStats()
{
    // if (SessionManager.SocietyID == 0) return;
    int sid = SessionManager.SocietyID;
    lblMemberCount.Text = DatabaseHelper.ExecuteScalar("SELE
```

- c. AssignTasksForm: Swapping Title and Description parameters while insert query.

```
DatabaseHelper.ExecuteNonQuery(
    @"INSERT INTO Tasks (SocietyID, AssignedToUserID, Assign
    VALUES (@S, @To, @By, @T, @D, @DD)",
    new SqlParameter("@S", SessionManager.SocietyID),
    new SqlParameter("@To", (int)cmbMember.SelectedValue),
    new SqlParameter("@By", SessionManager.UserID),
    new SqlParameter("@T", txtTaskDesc.Text.Trim()),
    new SqlParameter("@D", txtTaskTitle.Text.Trim()),
    new SqlParameter("@DD", dtpDue.Value));
```

- d. ManageMembersForm: Changing membership approval to status to "Pending" from "Approved".

```
private void btnApprove_Click(object sender, EventArgs e)

    if (dgvPending.SelectedRows.Count == 0) return;
    int id = Convert.ToInt32(dgvPending.SelectedRows[0].Cells["MembershipID"].Value);
    DatabaseHelper.ExecuteNonQuery("UPDATE Memberships SET Status='Pending', ApprovedAt=C
    MessageBox.Show("Membership approved!", "Success", MessageBoxButtons.OK, MessageBoxIcon
    LoadPending(); LoadApproved();
```

- e. AssignTasksForm: Changing task selection check to > 1 from ==0, bypassing single-selection safety.

```
private void btnMarkComplete_Click(object sender, EventArgs e)
{
    if (dgvTasks.SelectedRows.Count > 1) return;
    int id = Convert.ToInt32(dgvTasks.SelectedRows[0].Cells["TaskID"].Value)
    DatabaseHelper.ExecuteNonQuery("UPDATE Tasks SET Status='Completed' WHERE
    LoadTasks();
}
```

4) Student Module

- a. StudentDashboard: Changing Membership stats query to count "Pending" instead of "Approved".

```

ery("SELECT COUNT(*) FROM Memberships WHERE UserID = @UID AND Status='Pending'",
parameter("@UID", SessionManager.UserID));
[0][0].ToString());

ery("SELECT COUNT(*) FROM EventRegistrations WHERE UserID = @UID",
parameter("@UID", SessionManager.UserID));
].ToString());

ery("SELECT COUNT(*) FROM Memberships WHERE UserID = @UID AND Status='Pending'",
parameter("@UID", SessionManager.UserID));

```

- b. UpcomingEventsForm: Removing duplicate event registration check to allow multi-registration.

```

new SqlParameter("@EID", eventId), new SqlParameter("@UID", SessionManager.UserID)
// if (Convert.ToInt32(exists) > 0) { MessageBox.Show("You are already registered.")

```

- c. BrowseSocietiesForm: Changing applied society status to "Approved" instead of "Pending", bypassing approval.

```

nt result = DatabaseHelper.ExecuteNonQuery(
    "INSERT INTO Memberships (UserID, SocietyID, Status) VALUES (@UID, @SID, 'Approved')
    new SqlParameter("@UID", SessionManager.UserID), new SqlParameter("@SID", societyId)

```

- d. MyMembershipsForm: Removing UserID filter from query, exposing all students' memberships.

```

var dt = DatabaseHelper.ExecuteQuery(
    @"SELECT m.MembershipID, s.Name AS Society, s.Category, m.Status, m.AppliedAt
    FROM Memberships m INNER JOIN Societies s ON m.SocietyID = s.SocietyID
    WHERE m.UserID = m.UserID ORDER BY m.AppliedAt DESC",
    new System.Data.SqlClient.SqlParameter("@UID", SessionManager.UserID));

```

- e. BrowseSocietiesForm: Changing society browse query to load "Deleted" societies instead of "Approved".

```

ate void LoadSocieties()

string search = txtSearch.Text.Trim();
string query = @"SELECT s.SocietyID, s.Name, s.Category, s.Des
FROM Societies s LEFT JOIN Users u ON s.HeadU
WHERE s.Status = 'Deleted'";
if (!string.IsNullOrEmpty(search))
    query = " AND (s.Name LIKE @Search OR s.Category LIKE @Search)";

```

5) Core & Helper Module

- a. SessionManager: Commenting out SocietyID wipe on logout causing to leak state between users.

```

public static void Logout()
{
    UserID = 0;
    Username = null;
    FullName = null;
    Role = null;
    // SocietyID = 0;
    SocietyName = null;
}

```

- b. SessionManager: Assigned FullName to Username during login causing to display incorrect name.

```

public static void Login(int userId, s
{
    UserID = userId;
    Username = username;
    FullName = username;
    Role = role;
}

```

- c. DatabaseHelper: Changing ExecuteScalar to return ExecuteNonQuery to break aggregate functions.


```

public static object ExecuteScalar(string query, params SqlParameter[] parameters)
{
    using (SqlConnection conn = GetConnection())
    {
        using (SqlCommand cmd = new SqlCommand(query, conn))
        {
            if (parameters != null)
                cmd.Parameters.AddRange(parameters);

            conn.Open();
            return cmd.ExecuteNonQuery();
        }
    }
}

```

- d. DatabaseHelper: Changed ExecuteNonQuery to always return 1 to suppress SQL execution failures.

```

public static int ExecuteNonQuery(string query, params SqlParameter[] parameters)
{
    using (SqlConnection conn = GetConnection())
    {
        using (SqlCommand cmd = new SqlCommand(query, conn))
        {
            if (parameters != null)
                cmd.Parameters.AddRange(parameters);

            conn.Open();
            cmd.ExecuteNonQuery();
            return 1;
        }
    }
}

```

- e. LoginForm: Changed password hash generation to uppercase (X2) to lock out existing users.

```
public static string HashPassword(string password)
{
    using (SHA256 sha = SHA256.Create())
    {
        byte[] bytes = sha.ComputeHash(Encoding.UTF8.GetBytes(password));
        StringBuilder sb = new StringBuilder();
        foreach (byte b in bytes)
            sb.Append(b.ToString("X2"));
        return sb.ToString();
    }
}
```

Fault Injection Execution Results

Following are the results of existing testcases when run after the faults injection:

```
...\\Fault_Injected_Code > dotnet build SocietiesManagementSystem
m\\SocietiesManagementSystem.csproj
roj]
D:\\Documents\\Sem_8\\SMM\\Project\\Fault_Injected_Code\\SocietiesMa
nagementSystem\\Forms\\Student\\StudentDashboard.Designer.cs(11,6
3): warning CS0169: The field 'StudentDashboard.lblEventLabel'
is never used [D:\\Documents\\Sem_8\\SMM\\Project\\Fault_Injected_C
ode\\SocietiesManagementSystem\\SocietiesManagementSystem.cspro
j]
    3 Warning(s)
    0 Error(s)

Time Elapsed 00:00:04.50
```

```

..\Fault_Injected_Code > dotnet test SocietiesManagementSystem.Tests\SocietiesManagementSystem.Tests.csproj

1) at SocietiesManagementSystem.Tests.CoreTests.TC_CORE_02_SessionManager_LoginLogout() in D:\Documents\Sem_8\SMM\Project\Fault_Injected_Code\SocietiesManagementSystem.Tests\CoreTests.cs:line 39

Failed! - Failed: 3, Passed: 12, Skipped: 0, Total: 15, Duration: 45 s - SocietiesManagementSystem.Tests.dll (net48)

```

Results Summary:

- Authentication Module: 1 fault was detected by this module i.e., Mismatched password bypass caught by DB constraint.
- Admin Module: 1 fault was detected by this module i.e., Row selection index out of bounds.
- Core & Helper Module: 1 fault was detected by this module i.e., Session variable incorrect assignment.
- Society Module: No faults were detected in this module (Validation checks passed accidentally due to injected && logic).
- Student Module: No faults detected in this module.

Reliability Analysis Table

As not all artificial faults are found and no inherent faults were found ($i=0$), so the formula I'll use here is:

If not all artificial faults have been found ($f < A$), then:

$$\text{prob}(\text{no more than } E \text{ faults}) = \begin{cases} 0, & \text{if } i > E \\ \frac{\binom{A}{f-1}}{\binom{E+1+A}{E+f}}, & \text{if } i \leq E \end{cases}$$

Module / Feature	Total Faults Injected A	Number of Faults Identified	Probability (No more than E=1 fault)

		f	
Authentication Module	5	1	0.047 or 4.7%
Admin Module	5	1	0.047 or 4.7%
Core & Helper Module	5	1	0.047 or 4.7%
Society Module	5	0	0 or 0%
Student Module	5	0	0 or 0%

Most Reliable Module

According to above calculations, following are the most reliable modules with highest probability of having faults no more than 1:

1. Authentication Module
2. Admin Module
3. Core & Helper Module

Though overall probability of these module is very low but these values show that they have capability to detect the faults. Automated testcases which were derived from the cyclomatic complexity were able to find $f=1$ out of $A=5$ which included logical faults.

Least Reliable Module

According to above calculations, following are the least reliable modules with lowest probability of having faults no more than 1:

1. Society Module
2. Student Module

Automated testcases which were derived from the cyclomatic complexity were not able to find faults out of $A=5$ which included logical faults.

Student 2

Task 3

Justify which of the above generate module is the best one. Select a metric of your choice and defend your answer in the report (structural Metric).

Deliverable: Comparative table with list of all features and the value of the metrics for each of the feature.

Assumption: I have counted multiple calls of same function in a single function once.

Module 1: Admin

Class	Function	Fan-In	Fan-Out
AdminDashboard	AdminDashboard_Load	0	1
	LoadStats	4	1
	btnStudents_Click	0	1
	btnSocieties_Click	0	1
	btnEvents_Click	0	1
	btnReports_Click	0	0
	btnLogout_Click	0	1
ApproveEventsForm	ApproveEventsForm_Load	0	1
	LoadEvents	4	1
	btnApprove_Click	0	2
	btnReject_Click	0	2
	btnClose_Click	0	0
ManageSocietiesForm	ManageSocietiesForm_Load	0	1
	LoadSocieties	2	1
	UpdateStatus	3	2
	btnApprove_Click	0	1
	btnSuspend_Click	0	1
	btnDelete_Click	0	1
	btnClose_Click	0	0
ManageStudentsForm	ManageStudentsForm_Load	0	1

	LoadStudents	4	1
	btnSearch_Click	0	1
	btnActivate_Click	0	2
	btnDeactivate_Click	0	2
	btnClose_Click	0	0
UniversityReportsForm	btnStudentReport_Click	0	1
	btnSocietyReport_Click	0	1
	btnEventReport_Click	0	1
	btnMembershipReport_Click	0	1
	btnClose_Click	0	0
TOTAL		17	30

Module 2: Society

Class	Function	Fan-In	Fan-Out
AssignTasksForm	AssignTasksForm_Load	0	2
	LoadMembers	1	1
	LoadTasks	3	1
	btnAssign_Click	0	2
	btnMarkComplete_Click	0	2
	btnClose_Click	0	0
ManageEventsForm	ManageEventsForm_Load	0	1
	LoadEvents	3	1
	btnAdd_Click	0	3
	btnCancel_Click	0	2
	ClearFields	1	0

	btnClose_Click	0	0
ManageMembersForm	ManageMembersForm_Load	0	2
	LoadPending	3	1
	LoadApproved	4	1
	btnApprove_Click	0	3
	btnReject_Click	0	3
	btnRemove_Click	0	2
	btnClose_Click	0	0
SocietyDashboard	SocietyDashboard_Load	0	1
	LoadStats	3	1
	btnMembers_Click	0	1
	btnEvents_Click	0	1
	btnTasks_Click	0	0
	btnReports_Click	0	0
	btnLogout_Click	0	1
SocietyReportsForm	btnMemberReport_Click	0	1
	btnEventReport_Click	0	1
	btnClose_Click	0	0
TOTAL		18	34

Module 3: Student

Class	Function	Fan-In	Fan-Out
BrowseSocietiesForm	BrowseSocietiesForm_Load	0	1
	LoadSocieties	2	1

	btnSearch_Click	0	1
	btnApply_Click	0	2
	btnClose_Click	0	0
MyMembershipsForm	MyMembershipsForm_Load	0	1
	LoadData	1	1
	btnClose_Click	0	0
MyTicketsForm	MyTicketsForm_Load	0	1
	btnClose_Click	0	0
StudentDashboard	StudentDashboard_Load	0	1
	LoadStats	3	1
	btnBrowse_Click	0	1
	btnMemberships_Click	0	0
	btnEvents_Click	0	1
	btnTickets_Click	0	0
	btnLogout_Click	0	1
UpcomingEventsForm	UpcomingEventsForm_Load	0	1
	LoadEvents	2	1
	btnRegister_Click	0	3
	btnClose_Click	0	0
TOTAL		8	18

Module 4: Forms

Class	Function	Fan-In	Fan-Out
LoginForm	btnLogin_Click	0	3

	InkRegister_LinkClicked	0	0
	HashPassword	2	0
RegisterForm	btnRegister_Click	0	3
	InkLogin_LinkClicked	0	0
TOTAL		2	6

Modules 5: Helpers

Class	Function	Fan-In	Fan-Out
DatabaseHelper	GetConnection	3	0
	ExecuteQuery	24	1
	ExecuteNonQuery	15	1
	ExecuteScalar	10	1
SessionManager	Login	1	0
	SetSociety	1	0
	Logout	3	0
TOTAL		57	3

Comparison Table

Comparison Criteria	Admin Module	Society Module	Student Module	Forms Module	Helpers Module
Fan-In	17	18	8	2	57
Fan-In Evaluation	Very high reuse (>10)	Very high reuse (>10)	Good reuse (5–10)	Normal usage (1–5)	Very high reuse (>10)
Fan-Out	30	34	18	6	3

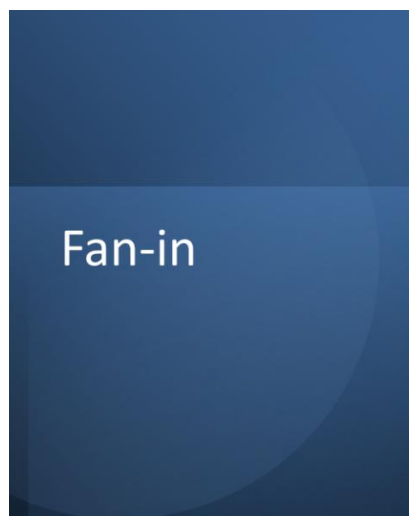
Fan-Out Evaluation	Poor design / too many responsibilities (>10)	Poor design / too many responsibilities (>10)	Poor design / too many responsibilities (>10)	Acceptable (4–7)	Good design (1–3)
Overall Evaluation	Highly dependent and complex	Most complex module	Moderately complex	Simple and manageable	Best module as it is highly reuse with very low dependency

Conclusion:

After evaluating the modules I have come to conclusion that Helpers module is the best one as that module does not depend much on other modules as it's fan-out is 3, and it's fan-in is 57 that means other modules are highly dependent on this module.

And if we consider the three main modules that are admin, society and student. Then, from these three modules Student is the best one as it has lower fan-out that means it does not depend on other modules if we compare to other two. But it's fan-in is less as well that means few depend on this module. Still it is much better than other two modules

Evaluation is done according to the metrics given in lecture slides:



Fan-in Value	Meaning
0	Module is never used (dead code or entry function)
1 – 5	Normal usage
5 – 10	Good reuse
>10	Very high reuse (may become risky if change impacts many modules)

Fan-out

Fan-out Value	Meaning
1 – 3	Good design
4 – 7	Acceptable
>7	Complex module
>10	Poor design / too many responsibilities

Task 4: Apply CK metrics on the complete generated project.

WMC – Weighted Methods per Class

Approach:

I am going to count the number of functions in each class and giving them CC 1.

Class	Methods	WMC	Interpretation
Program	Main()	1	Good
DatabaseHelper	GetConnection(), ExecuteQuery(), ExecuteNonQuery(), ExecuteScalar()	4	Good
SessionManager	Login(), SetSociety(), Logout()	3	Good
LoginForm	LoginForm(), btnLogin_Click(), lnkRegister_LinkClicked(), HashPassword()	4	Good
RegisterForm	RegisterForm(), btnRegister_Click(), lnkLogin_LinkClicked()	3	Good
AdminDashboard	AdminDashboard(), AdminDashboard_Load(),	8	Good

	LoadStats(), btnStudents_Click(), btnSocieties_Click(), btnEvents_Click(), btnReports_Click(), btnLogout_Click()		
ApproveEventsForm	ApproveEventsForm(), ApproveEventsForm_Load(), LoadEvents(), btnApprove_Click(), btnReject_Click(), btnClose_Click()	6	Good
ManageSocietiesForm	ManageSocietiesForm(), ManageSocietiesForm_Load() , LoadSocieties(), UpdateStatus(), btnApprove_Click(), btnSuspend_Click(), btnDelete_Click(), btnClose_Click()	8	Good
ManageStudentsForm	ManageStudentsForm(), ManageStudentsForm_Load() , LoadStudents(), btnSearch_Click(), btnActivate_Click(), btnDeactivate_Click(), btnClose_Click()	7	Good
UniversityReportsForm	UniversityReportsForm(), btnStudentReport_Click(), btnSocietyReport_Click(), btnEventReport_Click(), btnMembershipReport_Click() , btnClose_Click()	6	Good
AssignTasksForm	AssignTasksForm(), AssignTasksForm_Load(),	7	Good

	LoadMembers(), LoadTasks(), btnAssign_Click(), btnMarkComplete_Click(), btnClose_Click()		
ManageEventsForm	ManageEventsForm(), ManageEventsForm_Load(), LoadEvents(), btnAdd_Click(), btnCancel_Click(), ClearFields(), btnClose_Click()	7	Good
ManageMembersForm	ManageMembersForm(), ManageMembersForm_Load() , LoadPending(), LoadApproved(), btnApprove_Click(), btnReject_Click(), btnRemove_Click(), btnClose_Click()	8	Good
SocietyDashboard	SocietyDashboard(), SocietyDashboard_Load(), LoadStats(), btnMembers_Click(), btnEvents_Click(), btnTasks_Click(), btnReports_Click(), btnLogout_Click()	8	Good
SocietyReportsForm	SocietyReportsForm(), btnMemberReport_Click(), btnEventReport_Click(), btnClose_Click()	4	Good
BrowseSocietiesForm	BrowseSocietiesForm(), BrowseSocietiesForm_Load(), LoadSocieties(), btnSearch_Click(),	6	Good

	btnApply_Click(), btnClose_Click()		
MyMembershipsForm	MyMembershipsForm(), MyMembershipsForm_Load(), LoadData(), btnClose_Click()	4	Good
MyTicketsForm	MyTicketsForm(), MyTicketsForm_Load(), btnClose_Click()	3	Good
StudentDashboard	StudentDashboard(), StudentDashboard_Load(), LoadStats(), btnBrowse_Click(), btnMemberships_Click(), btnEvents_Click(), btnTickets_Click(), btnLogout_Click()	8	Good
UpcomingEventsForm	UpcomingEventsForm(), UpcomingEventsForm_Load() , LoadEvents(), btnRegister_Click(), btnClose_Click()	5	Good
LoginForm.Designer	Dispose(), InitializeComponent()	2	Good
RegisterForm.Designer	Dispose(), InitializeComponent(), CreateLabel(), CreateTextBox()	4	Good
AdminDashboard.Designer	Dispose(), InitializeComponent(), NavBtn(), Card()	4	Good
ApproveEventsForm.Designer	Dispose(), InitializeComponent()	2	Good

ManageSocietiesForm.Designer	Dispose(), InitializeComponent()	2	Good
ManageStudentsForm.Designer	Dispose(), InitializeComponent()	2	Good
UniversityReportsForm.Designer	Dispose(), InitializeComponent()	2	Good
AssignTasksForm.Designer	Dispose(), InitializeComponent()	2	Good
ManageEventsForm.Designer	Dispose(), InitializeComponent()	2	Good
ManageMembersForm.Designer	Dispose(), InitializeComponent(), CreateGrid()	3	Good
SocietyDashboard.Designer	Dispose(), InitializeComponent(), NavBtn(), Card()	4	Good
SocietyReportsForm.Designer	Dispose(), InitializeComponent()	2	Good
BrowseSocietiesForm.Designer	Dispose(), InitializeComponent()	2	Good
MyMembershipsForm.Designer	Dispose(), InitializeComponent()	2	Good
MyTicketsForm.Designer	Dispose(), InitializeComponent()	2	Good
StudentDashboard.Designer	Dispose(), InitializeComponent(), CreateNavButton(), CreateStatCard()	4	Good
UpcomingEventsForm.Designer	Dispose(), InitializeComponent()	2	Good

Metric:

According to the metric given in the lecture slides:

WMC Value	Interpretation
1 – 10	Good
10 – 20	Moderate complexity
> 20	High complexity

DIT – Depth of Inheritance Tree

All of the classes of forms inherits from the Form, every form class has DIT 1. While designer.cs classes does not inherits from any other class so they have DIT 0. Same is the case for helper, login, register and session manager files.

Class	DIT	Interpretation
Program	0	Simple hierarchy
DatabaseHelper	0	Simple hierarchy
SessionManager	0	Simple hierarchy
LoginForm	1	Simple hierarchy
RegisterForm	1	Simple hierarchy
AdminDashboard	1	Simple hierarchy
ApproveEventsForm	1	Simple hierarchy

ManageSocietiesForm	1	Simple hierarchy
ManageStudentsForm	1	Simple hierarchy
UniversityReportsForm	1	Simple hierarchy
AssignTasksForm	1	Simple hierarchy
ManageEventsForm	1	Simple hierarchy
ManageMembersForm	1	Simple hierarchy
SocietyDashboard	1	Simple hierarchy
SocietyReportsForm	1	Simple hierarchy
BrowseSocietiesForm	1	Simple hierarchy
MyMembershipsForm	1	Simple hierarchy
MyTicketsForm	1	Simple hierarchy
StudentDashboard	1	Simple hierarchy
UpcomingEventsForm	1	Simple hierarchy
LoginForm.Designer	0	Simple hierarchy
RegisterForm.Designer	0	Simple hierarchy
AdminDashboard.Designer	0	Simple hierarchy
ApproveEventsForm.Designer	0	Simple hierarchy
ManageSocietiesForm.Designer	0	Simple hierarchy
ManageStudentsForm.Designer	0	Simple hierarchy
UniversityReportsForm.Designer	0	Simple hierarchy
AssignTasksForm.Designer	0	Simple hierarchy
ManageEventsForm.Designer	0	Simple hierarchy
ManageMembersForm.Designer	0	Simple hierarchy
SocietyDashboard.Designer	0	Simple hierarchy
SocietyReportsForm.Designer	0	Simple hierarchy

BrowseSocietiesForm.Designer	0	Simple hierarchy
MyMembershipsForm.Designer	0	Simple hierarchy
MyTicketsForm.Designer	0	Simple hierarchy
StudentDashboard.Designer	0	Simple hierarchy
UpcomingEventsForm.Designer	0	Simple hierarchy

Metric:

According to the metric given in the lecture slides:

DIT	Meaning
0 – 2	Simple hierarchy
3 – 5	Moderate
> 5	Too deep

NOC – Number of Children

All form classes only inherits the Form of framework. Within the project there is no inheritance going on so there are no immediate children of any class in the project.

Class	NOC	Interpretation
Program	0	Leaf class
DatabaseHelper	0	Leaf class
SessionManager	0	Leaf class
LoginForm	0	Leaf class
RegisterForm	0	Leaf class

AdminDashboard	0	Leaf class
ApproveEventsForm	0	Leaf class
ManageSocietiesForm	0	Leaf class
ManageStudentsForm	0	Leaf class
UniversityReportsForm	0	Leaf class
AssignTasksForm	0	Leaf class
ManageEventsForm	0	Leaf class
ManageMembersForm	0	Leaf class
SocietyDashboard	0	Leaf class
SocietyReportsForm	0	Leaf class
BrowseSocietiesForm	0	Leaf class
MyMembershipsForm	0	Leaf class
MyTicketsForm	0	Leaf class
StudentDashboard	0	Leaf class
UpcomingEventsForm	0	Leaf class
LoginForm.Designer	0	Leaf class
RegisterForm.Designer	0	Leaf class
AdminDashboard.Designer	0	Leaf class
ApproveEventsForm.Designer	0	Leaf class
ManageSocietiesForm.Designer	0	Leaf class
ManageStudentsForm.Designer	0	Leaf class
UniversityReportsForm.Designer	0	Leaf class
AssignTasksForm.Designer	0	Leaf class
ManageEventsForm.Designer	0	Leaf class
ManageMembersForm.Designer	0	Leaf class

SocietyDashboard.Designer	0	Leaf class
SocietyReportsForm.Designer	0	Leaf class
BrowseSocietiesForm.Designer	0	Leaf class
MyMembershipsForm.Designer	0	Leaf class
MyTicketsForm.Designer	0	Leaf class
StudentDashboard.Designer	0	Leaf class
UpcomingEventsForm.Designer	0	Leaf class

Metric:

According to the metric given in the lecture slides:

NOC	Meaning
0	Leaf class
1 – 3	Good reuse
> 6	Possibly too generalized

CBO – Coupling Between Objects

CBO measures how many other classes a class is connected to. Connections occur through:

- Method calls
- Attribute references
- Inheritance
- Parameter types

The designer classes are not coupled with any other class in the project other than framework classes that I have skipped as they are not part of the modules. So their CBO is 0 and the remaining are evaluated as follows:

Class	Coupled Classes	CBO	Interpretation
Program	LoginForm	1	Low coupling
DatabaseHelper	None	0	Low coupling
SessionManager	None	0	Low coupling
LoginForm	DatabaseHelper, SessionManager, AdminDashboard, SocietyDashboard, StudentDashboard, RegisterForm	6	High coupling
RegisterForm	DatabaseHelper, LoginForm	2	Low coupling
AdminDashboard	DatabaseHelper, SessionManager, ManageStudentsForm, ManageSocietiesForm, ApproveEventsForm, UniversityReportsForm	6	High coupling
ApproveEventsForm	DatabaseHelper	1	Low coupling
ManageSocietiesForm	DatabaseHelper	1	Low coupling
ManageStudentsForm	DatabaseHelper	1	Low coupling
UniversityReportsForm	DatabaseHelper	1	Low coupling
AssignTasksForm	DatabaseHelper, SessionManager	2	Low coupling
ManageEventsForm	DatabaseHelper, SessionManager	2	Low coupling
ManageMembersForm	DatabaseHelper, SessionManager	2	Low coupling
SocietyDashboard	DatabaseHelper, SessionManager, ManageMembersForm, ManageEventsForm, AssignTasksForm, SocietyReportsForm	6	High coupling
SocietyReportsForm	DatabaseHelper, SessionManager	2	Low coupling
BrowseSocietiesForm	DatabaseHelper, SessionManager	2	Low coupling
MyMembershipsForm	DatabaseHelper, SessionManager	2	Low coupling
MyTicketsForm	DatabaseHelper, SessionManager	2	Low coupling
StudentDashboard	DatabaseHelper, SessionManager, BrowseSocietiesForm, MyMembershipsForm, UpcomingEventsForm, MyTicketsForm	6	High coupling
UpcomingEventsForm	DatabaseHelper, SessionManager	2	Low coupling
LoginForm.Designer	None	0	Low coupling
RegisterForm.Designer	None	0	Low coupling
AdminDashboard.Designer	None	0	Low coupling
ApproveEventsForm.Designer	None	0	Low coupling
ManageSocietiesForm.Designer	None	0	Low coupling

ManageStudentsForm.Designer	None	0	Low coupling
UniversityReportsForm.Designer	None	0	Low coupling
AssignTasksForm.Designer	None	0	Low coupling
ManageEventsForm.Designer	None	0	Low coupling
ManageMembersForm.Designer	None	0	Low coupling
SocietyDashboard.Designer	None	0	Low coupling
SocietyReportsForm.Designer	None	0	Low coupling
BrowseSocietiesForm.Designer	None	0	Low coupling
MyMembershipsForm.Designer	None	0	Low coupling
MyTicketsForm.Designer	None	0	Low coupling
StudentDashboard.Designer	None	0	Low coupling
UpcomingEventsForm.Designer	None	0	Low coupling

Metric:

According to the metric given in the lecture slides:

CBO	Meaning
0 – 3	Low coupling
4 – 6	Moderate
> 6	High coupling

RFC – Response for Class

RFC is the number of methods that can potentially execute in response to a message received by an object.

Formula:

$$\text{RFC} = \text{Methods in class} + \text{Remote Methods called by those methods}$$

Similar to my previous approaches I am skipping the framework function callings. Only those that are present in the feature classes of the modules. Also in this CK-Suit metric we also skip the constructors. So I have also skip[ped those.

Class Name	Unique Methods + Remote Calls	RFC	Evaluation
LoginForm	btnLogin_Click, lnkRegister_LinkClicked, HashPassword, ExecuteQuery, Login, SetSociety	6	Moderate complexity
RegisterForm	btnRegister_Click, lnkLogin_LinkClicked, ExecuteScalar, ExecuteNonQuery, HashPassword	5	Moderate complexity
AdminDashboard	AdminDashboard_Load, LoadStats, btnStudents_Click, btnSocieties_Click, btnEvents_Click, btnReports_Click, btnLogout_Click, FullName, ExecuteScalar, ManageStudentsForm, ManageSocietiesForm, ApproveEventsForm, UniversityReportsForm, Logout	14	High complexity
ApproveEventsForm	ApproveEventsForm_Load, LoadEvents, btnApprove_Click, btnReject_Click, btnClose_Click, ExecuteQuery, ExecuteNonQuery	7	Moderate complexity
ManageSocietiesForm	ManageSocietiesForm_Load, LoadSocieties, UpdateStatus, btnApprove_Click, btnSuspend_Click, btnDelete_Click, btnClose_Click, ExecuteQuery, ExecuteNonQuery	9	Moderate complexity
ManageStudentsForm	ManageStudentsForm_Load, LoadStudents, btnSearch_Click, btnActivate_Click, btnDeactivate_Click, btnClose_Click, ExecuteQuery, ExecuteNonQuery	8	Moderate complexity

UniversityReportsForm	btnStudentReport_Click, btnSocietyReport_Click, btnEventReport_Click, btnMembershipReport_Click, btnClose_Click, ExecuteQuery	6	Moderate complexity
AssignTasksForm	AssignTasksForm_Load, LoadMembers, LoadTasks, btnAssign_Click, btnMarkComplete_Click, btnClose_Click, ExecuteQuery, ExecuteNonQuery	8	Moderate complexity
ManageEventsForm	ManageEventsForm_Load, LoadEvents, btnAdd_Click, btnCancel_Click, ClearFields, btnClose_Click, ExecuteQuery, ExecuteNonQuery	8	Moderate complexity
ManageMembersForm	ManageMembersForm_Load, LoadPending, LoadApproved, btnApprove_Click, btnReject_Click, btnRemove_Click, btnClose_Click, ExecuteQuery, ExecuteNonQuery	9	Moderate complexity
SocietyDashboard	SocietyDashboard_Load, LoadStats, btnMembers_Click, btnEvents_Click, btnTasks_Click, btnReports_Click, btnLogout_Click, FullName, SocietyName, ExecuteScalar, SocietyID, ManageMembersForm, ManageEventsForm, AssignTasksForm, SocietyReportsForm, Logout	16	High complexity
SocietyReportsForm	btnMemberReport_Click, btnEventReport_Click, btnClose_Click, ExecuteQuery, SocietyID, SocietyName	6	Moderate complexity
BrowseSocietiesForm	BrowseSocietiesForm_Load, LoadSocieties, btnSearch_Click, btnApply_Click, btnClose_Click, ExecuteQuery, ExecuteScalar, ExecuteNonQuery, UserID	9	Moderate complexity

MyMembershipsForm	MyMembershipsForm_Load, LoadData, btnClose_Click, ExecuteQuery, UserID	5	Moderate complexity
MyTicketsForm	MyTicketsForm_Load, btnClose_Click, ExecuteQuery, UserID	4	Low complexity
StudentDashboard	StudentDashboard_Load, LoadStats, btnBrowse_Click, btnMemberships_Click, btnEvents_Click, btnTickets_Click, btnLogout_Click, FullName, ExecuteQuery, UserID, BrowseSocietiesForm, MyMembershipsForm, UpcomingEventsForm, MyTicketsForm, Logout	15	High complexity
UpcomingEventsForm	UpcomingEventsForm_Load, LoadEvents, btnRegister_Click, btnClose_Click, ExecuteQuery, ExecuteScalar, ExecuteNonQuery, UserID	8	Moderate complexity
DatabaseHelper	GetConnection, ExecuteQuery, ExecuteNonQuery, ExecuteScalar	4	Low complexity
SessionManager	Login, SetSociety, Logout	3	Low complexity
Program	Main, LoginForm	2	Low complexity

Metric:

According to the metric given in the lecture slides:

- High RFC means:
 - High complexity
 - Difficult testing
 - Large communication

As this metric isn't quantized so I use the following according to my understanding:

RFC Range	Evaluation
1 – 4	Low complexity
5 – 10	Moderate complexity
> 10	High complexity

LCOM – Lack of Cohesion in Methods

For LCOM what I have done is I only did it for the business logic and not for variables that are part of framework such as UI controls like textbox, datagridview, etc. Also the methods such as initializecomponent() or dispose() are excluded.

Only two classes have functions that share variables and in the remaining no variables are defined. So in conclusion design is good.

Concept of LCOM:

P = Number of method pairs with no shared instance variables

Q = Number of method pairs with shared instance variables

When, $P > Q$ then $LCOM = P - Q$. Else, it is 0.

Evaluation:

Class	Business State Variables	Variable Sharing / Methods	Business Methods	P	Q	LCOM	Interpretation

		Using Variables					
DatabaseHelper	ConnectionString	Connection String → GetConnection(), ExecuteQuery(), ExecuteNonQuery(), ExecuteScalar()	GetConnection(), ExecuteQuery(), ExecuteNonQuery(), ExecuteScalar()	0	6	0	Good design
SessionManager	UserID, Username, FullName, Role, SocietyID, SocietyName	UserID → Login(), Logout() Username → Login(), Logout() FullName → Login(), Logout() Role → Login(), Logout() SocietyID → SetSociety(), Logout() SocietyName → SetSociety(), Logout()	Login(), SetSociety(), Logout()	1	2	0	Good design
LoginForm	None	None	btnLogin_Click(), lnkRegister_LinkClicked(), HashPassword()	0	0	0	Good design

RegisterForm	None	None	btnRegister_Click() , lnkLogin_LinkClicked()	0	0	0	Good design
AdminDashboard	None	None	AdminDashboard_Load(), LoadStats(), btnStudents_Click(), btnSocieties_Click(), btnEvents_Click(), btnReports_Click(), btnLogout_Click()	0	0	0	Good design
ApproveEventsForm	None	None	ApproveEventsForm_Load(), LoadEvents(), btnApprove_Click(), , btnReject_Click()	0	0	0	Good design
ManageSocietiesForm	None	None	ManageSocietiesForm_Load(), LoadSocieties(), UpdateStatus(), btnApprove_Click(), , btnSuspend_Click(), btnDelete_Click()	0	0	0	Good design
ManageStudentsForm	None	None	ManageStudentsForm_Load(), LoadStudents(), btnSearch_Click(), btnActivate_Click(), , btnDeactivate_Click()	0	0	0	Good design

UniversityReportsForm	None	None	btnStudentReport_Click(), btnSocietyReport_Click(), btnEventReport_Click(), btnMembershipReport_Click()	0	0	0	Good design
AssignTasksForm	None	None	AssignTasksForm_Load(), LoadMembers(), LoadTasks(), btnAssign_Click(), btnMarkComplete_Click()	0	0	0	Good design
ManageEventsForm	None	None	ManageEventsForm_Load(), LoadEvents(), btnAdd_Click(), btnCancel_Click(), ClearFields()	0	0	0	Good design
ManageMembersForm	None	None	ManageMembersForm_Load(), LoadPending(), LoadApproved(), btnApprove_Click(), btnReject_Click(), btnRemove_Click()	0	0	0	Good design
SocietyDashboard	None	None	SocietyDashboard_Load(), LoadStats(), btnMembers_Click(), btnEvents_Click(), btnTasks_Click(),	0	0	0	Good design

			btnReports_Click(), btnLogout_Click()				
SocietyReportsForm	None	None	btnMemberReport_Click(), btnEventReport_Click()	0	0	0	Good design
BrowseSocietiesForm	None	None	BrowseSocietiesForm_Load(), LoadSocieties(), btnSearch_Click(), btnApply_Click()	0	0	0	Good design
MyMembershipsForm	None	None	MyMembershipsForm_Load(), LoadData()	0	0	0	Good design
MyTicketsForm	None	None	MyTicketsForm_Load()	0	0	0	Good design
StudentDashboard	None	None	StudentDashboard_Load(), LoadStats(), btnBrowse_Click(), btnMemberships_Click(), btnEvents_Click(), btnTickets_Click(), btnLogout_Click()	0	0	0	Good design
UpcomingEventsForm	None	None	UpcomingEventsForm_Load(), LoadEvents(), btnRegister_Click()	0	0	0	Good design

Metric:

According to the metric given in the lecture slides:

LCOM	Meaning
Low	Good design
High	Poor cohesion

Analysis summary and findings

1. *Maximum Depth of Inheritance (DIT)*

The maximum DIT in the project is **1**. All form classes inherit directly from the base Form class provided by the framework. Helper classes (DatabaseHelper, SessionManager), Program, and designer classes have DIT = 0. There is no deeper inheritance hierarchy anywhere in the project, indicating a very flat and simple inheritance structure.

2. *Highest and Lowest WMC*

Highest WMC: AdminDashboard, ManageSocietiesForm, ManageMembersForm, and SocietyDashboard all have 8 methods having WMC=8. These dashboard and management classes contain multiple event handlers, load methods, and navigation logic.

Lowest WMC: Several classes like MyTicketsForm, Program, and various .Designer classes have 2–3 methods. They perform minimal, focused tasks for example basic form loading or disposal. All classes fall in the good range per the provided interpretation.

3. *Class with the greatest number of children (NOC)*

No class has any children having NOC = 0 for all classes. The project has no custom inheritance relationships between user-defined classes. All forms only inherit from the framework's Form base class, making every class a leaf class.

4. *Most complex class*

SocietyDashboard is the most complex. It has high WMC that is 8, high RFC that is 16, and high CBO that is 6. It coordinates multiple forms such as ManageMembersForm,

ManageEventsForm, etc. It handles session data, database calls, and UI navigation. This central control point accumulates many responsibilities, leading to higher complexity.

5. Most coupled class

LoginForm, AdminDashboard, SocietyDashboard, and StudentDashboard that is CBO=6 each. They are tightly coupled because they interact with multiple other classes: DatabaseHelper, SessionManager, and several form classes for navigation. LoginForm especially acts as an entry point that branches to all dashboards, while dashboards launch multiple management/report forms.

6. Least cohesive class

All classes show good cohesion that is LCOM=0. Only DatabaseHelper and SessionManager have shared state variables, but their methods are logically related and properly use those variables. No class has methods that do not share variables when they should, or unrelated methods. The design avoids god classes with unrelated responsibilities.

Task 9 : Refactoring Suggestions for the Student Management System

1. Helper module

For the Helpers Module, the primary concern is the extremely high Fan-In values on core database methods like ExecuteQuery (24), ExecuteNonQuery (15), and ExecuteScalar (10). These functions act as a central bottleneck, making the entire application tightly coupled and difficult to maintain or test. Refactoring should focus on introducing a Repository pattern or dedicated service classes for each entity (StudentRepository, EventRepository, etc.). This will significantly reduce fan-in on low-level methods, improve type safety, and make error handling more consistent.

2. Forms module

LoginForm & RegisterForm requires immediate attention due to btnLogin_Click having a Cyclomatic Complexity of 8 and btnRegister_Click at 5. These high-complexity event handlers mix validation, hashing, database calls, and navigation logic, which reduces readability and testability. The suggestion is to extract all business logic into a clean AuthService class with separate methods for Login, Register, and HashPassword. This separation will lower cyclomatic complexity in UI handlers to 1-3 and allow better unit testing of authentication rules. It will also resolve issues like password mismatch bypasses and role assignment faults observed during fault injection.

Overall Architecture Recommendations

I recommend adopting a clean layered architecture (Presentation → Application Services → Data Access) across the entire application. Most forms currently call DatabaseHelper methods directly, which violates separation of concerns and increases coupling.

By introducing this layered structure along with dependency injection, I can significantly reduce cyclomatic complexity in UI handlers, lower fan-in/fan-out issues, and improve reliability against logical faults. This will also make the cyclomatic complexity-based test cases more effective in future fault injections.

Student 3

Task 7: Cocomo Model:

Suitable cocomo model will be basic cocomo model. We will only be considering .cs and .designer.cs files for the loc count and not include the config files or project info files. here are the line of code break down for the project:

.cs Files

File Name	Lines of Code
AdminDashboard.cs	28
ApproveEventsForm.cs	37
ManageSocietiesForm.cs	37
ManageStudentsForm.cs	42
UniversityReportsForm.cs	70
LoginForm.cs	93
RegisterForm.cs	55
AssignTasksForm.cs	58
ManageEventsForm.cs	49
ManageMembersForm.cs	55
SocietyDashboard.cs	31
SocietyReportsForm.cs	41
BrowseSocietiesForm.cs	64
MyMembershipsForm.cs	22
MyTicketsForm.cs	22
StudentDashboard.cs	54
UpcomingEventsForm.cs	40
DatabaseHelper.cs	62
SessionManager.cs	33
Program.cs	16
AssemblyInfo.cs	14

--	--

Total .cs Code Lines: 923

.Designer.cs Files

File Name	Lines of Code
AdminDashboard.Designer.cs	68
ApproveEventsForm.Designer.cs	50
ManageSocietiesForm.Designer.cs	53
ManageStudentsForm.Designer.cs	55
UniversityReportsForm.Designer.cs	57
LoginForm.Designer.cs	114
RegisterForm.Designer.cs	116
AssignTasksForm.Designer.cs	69
ManageEventsForm.Designer.cs	78
ManageMembersForm.Designer.cs	61
SocietyDashboard.Designer.cs	68
SocietyReportsForm.Designer.cs	50
BrowseSocietiesForm.Designer.cs	59
MyMembershipsForm.Designer.cs	46
MyTicketsForm.Designer.cs	46
StudentDashboard.Designer.cs	77
UpcomingEventsForm.Designer.cs	49

Total .Designer.cs Code Lines: 1116

Overall Project Code Summary

Category	Total Lines of Code
----------	---------------------

.cs Files	923
.Designer.cs Files	1116
Grand Total	2039

Effort: $a(KLOC)^b = 2.4(2.039)^{1.05} = 5.07PM$

it would take 1 person about 5.07 months of work to achieve this.

Duration: $c(Effort)^d = 2.5(5.07)^{0.38} = 4.6 \text{ Months}$

Persons: $ceil(Effort/Duration) = ceil(5.07/4.6) = ceil(1.10) = 2 \text{ Persons}$

Task 8: Documentation ratio:

Total comment = 27

Total LOC = 2039

Documentation Ratio = $2039/27 = 75.51$

So, for approximately 76 lines of code there is one comment. In conclusion it tells us that the document ratio is very bad which in turns means that cod maintainability and understanding might also be bad.

Task 6: KLM analysis:

Login Screen:

1. Mental preparation: 1350 ms
2. Move mouse to username: 1100 ms
3. Click username: 280 ms
4. Type username (6 letters): 1680 ms
5. Move mouse to password: 1100 ms
6. Click password: 280 ms
7. Move mouse back to keyboard: 400 ms
8. Type password (3 letters): 840 ms

Total = $1350 + 1100 + 280 + 1680 + 1100 + 280 + 400 + 840$

Total = 7,030 ms (or 7.03 seconds).

Admin Dashboard:

1. Mental preparation: 1350 ms
2. Move mouse to section: 1100 ms
3. Click section: 280 ms

Total = $1350 + 1100 + 280$

Total = 2,730 ms (or 2.73 seconds).

Manage students:

Search task:

1. Mental preparation: 1350 ms
2. Move mouse to search bar: 1100 ms
3. Click search bar: 280 ms
4. Move mouse to keyboard: 400 ms
5. Type name (4 letters): 1120 ms
6. Click search button: 280 ms

Total = $1350 + 1100 + 280 + 400 + 1120 + 280$

Total = 5,530 ms (or 5.53 seconds).

Activate/deactivate membership:

1. Mental preparation: 1350 ms
2. Move mouse to student: 1100 ms
3. Click on student: 280 ms
4. Move mouse to button: 1100 ms
5. Click button (keystroke): 280 ms

Total = $1350 + 1100 + 280 + 1100 + 280$

Total = 4,110 ms (or 4.11 seconds).

Manage Societies:

Approve/suspend/delete societies:

1. Mental preparation: 1350 ms
2. Move mouse to society: 1100 ms
3. Click on society: 280 ms
4. Move mouse to action button (approve/suspend/delete): 1100 ms
5. Click action button: 280 ms

Sum total = $1350 + 1100 + 280 + 1100 + 280$

Total = 4,110 ms (or 4.11 seconds).

Approve events:

1. Mental preparation: 1350 ms
2. Move mouse to event: 1100 ms

3. Click on event: 280 ms
4. Move mouse to approve/reject button: 1100 ms
5. Click approve/reject button: 280 ms

Sum total = $1350 + 1100 + 280 + 1100 + 280$

Total = 4,110 ms (or 4.11 seconds).

University Reports:

1. Mental preparation: 1350 ms
2. Move mouse to report: 1100 ms
3. Click on report: 280 ms

Sum total = $1350 + 1100 + 280$

Total = 2,730 ms (or 2.73 seconds).

Society Head Dashboard:

1. Mental preparation: 1350 ms
2. Move mouse to the target field: 1100 ms
3. Click on the field: 280 ms

Sum total = $1350 + 1100 + 280$

Total = 2,730 ms (or 2.73 seconds).

Manage Members:

Pending request:

1. Mental preparation: 1350 ms
2. Move mouse to student who applied: 1100 ms
3. Click on student: 280 ms
4. Move mouse to approve/reject button: 1100 ms
5. Click approve/reject button: 280 ms

Sum total = $1350 + 1100 + 280 + 1100 + 280$

Total = 4,110 ms (or 4.11 seconds).

Remove Members:

1. Mental preparation: 1350 ms

2. Move mouse to the student: 1100 ms
3. Click on student: 280 ms
4. Move mouse to remove button: 1100 ms
5. Click remove button: 280 ms

Sum total = 1350 + 1100 + 280 + 1100 + 280

Total = 4,110 ms (or 4.11 seconds).

Manage Events:

Add event:

- Mental preparation: 1350 ms
- Move to title: 1100 ms
- Click title: 280 ms
- Type title: 1400 ms
- Move to type field: 1100 ms
- Click type field: 280 ms
- Move to dropdown: 1100 ms
- Click dropdown: 280 ms
- Move to capacity: 1100 ms
- Click capacity: 280 ms
- Type capacity: 840 ms
- Move to description: 1100 ms
- Click description: 280 ms
- Type description: 5600 ms
- Move to venue: 1100 ms
- Click venue: 280 ms
- Type venue: 2800 ms
- Move to date: 1100 ms

- Click date: 280 ms
- Move to create button: 1100 ms
- Click create button: 280 ms

Total = 1350 + 1100 + 280 + 1400 + 1100 + 280 + 1100 + 280 + 1100 + 280 + 840 + 1100 + 280
+ 5600 + 1100 + 280 + 2800 + 1100 + 280 + 1100 + 280

Total = 22,780 ms (or 22.78 seconds).

Cancel event:

1. Mental preparation: 1350 ms
2. Move mouse to the event: 1100 ms
3. Click on the event: 280 ms
4. Move mouse to cancel button: 1100 ms
5. Click cancel button: 280 ms

Sum total = 1350 + 1100 + 280 + 1100 + 280

Total = 4,110 ms (or 4.11 seconds).

Assign tasks:

Assignment:

1. Mental preparation: 1350 ms
2. Move mouse to title field: 1100 ms
3. Click title field: 280 ms
4. Type title (5 letters): $5 \times 280 = 1400$ ms
5. Move mouse to assign-to field: 1100 ms
6. Click assign-to field: 280 ms
7. Move mouse to dropdown: 1100 ms
8. Click dropdown (select member): 280 ms
9. Move mouse to description field: 1100 ms
10. Click description field: 280 ms
11. Type description (20 letters): $20 \times 280 = 5600$ ms

12. Move mouse to date field: 1100 ms

13. Click date field: 280 ms

14. Move mouse to keyboard: 400 ms

15. Move mouse back to dropdown (keyboard → mouse): 400 ms

16. Move mouse to assign button: 1100 ms

17. Click assign button: 280 ms

Sum total = $1350 + 1100 + 280 + 1400 + 1100 + 280 + 1100 + 280 + 1100 + 280 + 5600 + 1100 + 280 + 400 + 400 + 1100 + 280$

Total = 17,230 ms (or 17.23 seconds).

Mark complete:

1. Mental preparation: 1350 ms

2. Move mouse to task: 1100 ms

3. Click on task: 280 ms

4. Move mouse to complete button: 1100 ms

5. Click complete button: 280 ms

Sum total = $1350 + 1100 + 280 + 1100 + 280$

Total = 4,110 ms (or 4.11 seconds).

Reports:

1. Mental preparation: 1350 ms

2. Move mouse to the report: 1100 ms

3. Click on the report: 280 ms

Sum total = $1350 + 1100 + 280$

Total = 2,730 ms (or 2.73 seconds).

Student Dashboard:

1. Mental preparation: 1350 ms

2. Move mouse to the target field: 1100 ms

3. Click on the field: 280 ms

Sum total = $1350 + 1100 + 280$

Total = 2,730 ms (or 2.73 seconds).

Browse Societies:

Search:

1. Mental preparation: 1350 ms
2. Move mouse to search bar: 1100 ms
3. Click on search bar: 280 ms
4. Type society name (5 letters): $5 \times 280 = 1400$ ms
5. Click search button: 280 ms

Sum total = $1350 + 1100 + 280 + 1400 + 280$

Total = 4,410 ms (or 4.41 seconds).

Apply to society:

1. Mental preparation: 1350 ms
2. Move mouse to the society: 1100 ms
3. Click on the society: 280 ms
4. Move mouse to apply button: 1100 ms
5. Click apply button: 280 ms

Sum total = $1350 + 1100 + 280 + 1100 + 280$

Total = 4,110 ms (or 4.11 seconds).

My Memberships:

1. Mental preparation: 1350 ms

Total = 1350 ms (or 1.35 seconds).

My Tickets:

1. Mental preparation: 1350 ms

Total = 1350 ms (or 1.35 seconds).

Upcoming Events:

1. Mental preparation: 1350 ms
2. Move mouse to the society: 1100 ms
3. Click on the society: 280 ms
4. Move mouse to apply button: 1100 ms
5. Click apply button: 280 ms

Sum total = $1350 + 1100 + 280 + 1100 + 280$

Total = 4,110 ms (or 4.11 seconds).

Sign up:

- Mental preparation: 1350 ms
- Move to name field: 1100 ms
- Click name field: 280 ms
- Move to keyboard: 400 ms
- Type name: 2800 ms
- Move to mouse: 400 ms
- Move to username field: 1100 ms
- Click username field: 280 ms
- Move to keyboard: 400 ms
- Type username: 2800 ms
- Move to mouse: 400 ms
- Move to email field: 1100 ms
- Click email field: 280 ms
- Move to keyboard: 400 ms
- Type email: 4200 ms
- Move to mouse: 400 ms
- Move to phone field: 1100 ms

- Click phone field: 280 ms
- Move to keyboard: 400 ms
- Type phone: 3640 ms
- Move to mouse: 400 ms
- Move to password field: 1100 ms
- Click password field: 280 ms
- Move to keyboard: 400 ms
- Type password: 2240 ms
- Move to mouse: 400 ms
- Move to confirm field: 1100 ms
- Click confirm field: 280 ms
- Move to keyboard: 400 ms
- Type confirm: 2240 ms
- Move to mouse: 400 ms
- Move to register button: 1100 ms
- Click register button: 280 ms

Total = 32,030 ms (or 32.03 seconds).